Hyperledger Fabric Framework for Residue-Safe Cardamom Supply Chains and Smart Payments

ANONYMOUS AUTHOR, Anonymous Institution, Anonymous Country

Permissioned blockchain networks such as Hyperledger Fabric are increasingly adopted in agricultural supply chains to ensure traceability, transparency, and automated financial settlements. However, as transaction loads and participant densities grow, these networks exhibit complex, non-linear performance degradation driven by Multi-Version Concurrency Control (MVCC) conflicts—a phenomenon that remains poorly understood and difficult to predict analytically. This study uses an agricultural supply chain system, implemented on Hyperledger Fabric with smart contract-driven auctions, role-based access control, IPFS-based off-chain storage, and automated payments, as a representative real-world case study to systematically investigate these scalability limits. The system was rigorously benchmarked using Hyperledger Caliper across a comprehensive matrix of transaction send rates, participant densities, and chaincode functions. The primary research objective is to develop a machine learning-based predictive framework—using eXtreme Gradient Boosting (XGBoost)—capable of accurately forecasting both transaction throughput and MVCC failure rates from static algorithmic complexity features and dynamic network metrics, without relying on categorical function identifiers. The trained XGBoost model demonstrated robust generalization on unseen network configurations, achieving a testing R^2 of 0.84 for throughput prediction and 0.82 for MVCC failure rate prediction. By analyzing information gain from the trained models, this study mathematically proves that MVCC hot-key participant density is the overwhelmingly dominant catalyst for transaction failures (75.0% predictive importance), while the number of ledger writes is the primary constraint on achievable throughput (30.0% importance). These findings provide a generalizable, interpretable framework for predicting and diagnosing performance bottlenecks in permissioned blockchain deployments.

ACM Reference Format:
Anonymous Author. 2026. Hyperledger Fabric Framework for Residue-Safe Cardamom Supply Chains and Smart Payments. In *Proceedings of Submitted manuscript (Submitted Manuscript).* ACM, New York, NY, USA, **??** pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Keywords

Hyperledger Fabric; MVCC conflict prediction; Blockchain performance analysis; XGBoost; Transaction throughput; Failure rate prediction; Smart contracts; Agricultural supply chain; Hyperledger Caliper; Permissioned blockchain

Author's Contact Information: Anonymous Author, Anonymous Institution, Anonymous City, Anonymous State, Anonymous Country.

1 Introduction

Permissioned blockchain frameworks, particularly Hyperledger Fabric, have emerged as a powerful foundation for building trustworthy, multi-stakeholder supply chain systems. Their ability to enforce immutable records, role-based access control, and automated smart contract logic makes them attractive for agri-food applications where traceability, transparent pricing, and automated financial settlements are essential. High-value agricultural supply chains represent a compelling case study for such a system: they involve fragmented stakeholder networks, opaque auction markets, and a pressing need for end-to-end lot traceability from farm to consumer. Farmers often receive unfair prices in regional auction centers where bids are opaque and collusion is common, frequently leading to severe payment defaults and systemic financial risk for producers. These challenges—lack of transparency, payment insecurity, and absence of produce provenance—motivate a blockchain-based platform that records every supply chain event immutably and automates financial settlements through smart contracts.

Although digital technologies such as Near Field Communication (NFC), Radio Frequency Identification (RFID), and Quick Response (QR) codes have been used for product tracking, they rely on centralized data stores that are vulnerable to tampering and single points of failure ([**?**]). Blockchain technology overcomes these limitations by providing a decentralized, tamper-proof shared ledger accessible to all authorized stakeholders. Hyperledger Fabric, in particular, has been adopted in numerous agri-food traceability initiatives by organizations such as the World Wide Fund (WWF) and the Food and Agriculture Organization of the United Nations (FAO) (**?**), yet comprehensive systems addressing complex auction and settlement concurrency remain scarce in regional agricultural supply chains.

This paper addresses this gap by proposing a Hyperledger Fabric-based agricultural supply chain platform that integrates end-to-end lot traceability, a smart contract-driven decentralized auction mechanism for transparent price discovery, automated financial settlements, role-based access control, IPFS-based off-chain storage, and consumer-facing packet-level provenance verification. Crucially, this specific supply chain architecture was selected because it serves a dual purpose: it addresses an urgent socio-economic need while providing an ideal technical testbed for performance modeling. Deploying such a platform—with its high-frequency auction bidding, concurrent payment settlements, and multiple participant roles—creates massive "hot-key" contention. This inherently exposes the network to severe Multi-Version Concurrency Control (MVCC) conflicts under load. In Hyperledger Fabric's optimistic concurrency model, if two transactions read the same ledger key version within the same block window, only one can commit; the rest are silently dropped as MVCC `READ_CONFLICT` failures. The complex business logic of agricultural auction use cases naturally generates the exact high-contention scenarios required to rigorously study and model these structural failure modes.

Predicting and diagnosing these MVCC failure rates and throughput ceilings analytically is challenging because the relationship between concurrent participant density, chaincode complexity, and failure probability is highly nonlinear. Existing literature on Hyperledger Fabric performance benchmarking primarily characterizes behavior under controlled settings and rarely develops generalizable predictive models. This study addresses this gap directly: the **primary objective** is to develop and validate a machine learning-based predictive framework—using eXtreme Gradient Boosting (XGBoost)—that can accurately forecast MVCC failure rates and transaction throughput from interpretable structural features (ledger reads/writes, hot-key participant density, send rate, and Caliper workers). A representative agricultural supply chain system serves as the empirical testbed, providing a rich dataset of benchmarked chaincode functions under a comprehensive matrix of transaction loads and participant configurations.

2 Related Work

Agri-food supply chains involve the movement of agricultural products from producers to consumers through several stakeholders, including farmers, processors, distributors, retailers, and consumers [??]. Blockchain technology has gained attention as a tool for improving trust, traceability, and accountability in these supply chains [??].

2.1 Hyperledger-Based Applications in Agriculture

Hyperledger Fabric, introduced by Androulaki et al. [?] as a modular, permissioned distributed ledger framework, has since been widely adopted for enterprise supply chain applications. Its execute-order-validate (EOV) architecture separates transaction simulation from consensus ordering, enabling parallel smart contract execution—but also introducing the MVCC conflict vulnerabilities that are the central focus of this study. In the agri-food domain, numerous architectures have been proposed to enhance transparency and integrate IoT data for monitoring agricultural products [????]. For example, Sugandh et al. [?] employed Hyperledger Fabric and Caliper to demonstrate risk traceability for agricultural products, achieving peak write throughput of 205.87 TPS in a two-organization configuration, directly establishing the benchmarking methodology adopted in our study. Other recent works have implemented dynamic pricing frameworks for seafood supply chains and halal certification systems in the UAE, leveraging Hyperledger Fabric with IPFS and Caliper benchmarking [???]. While these works collectively demonstrate the value of permissioned blockchains for trusted record-keeping, they primarily focus on historical data logging and logistics under moderate transaction loads. None develop a predictive analytical framework to anticipate MVCC failure rates or throughput ceilings—the concurrency bottlenecks that inevitably arise when deploying high-frequency smart contracts such as decentralized auctions or concurrent payment settlements.

2.2 Performance Benchmarking of Blockchain Systems

To understand the limitations of permissioned blockchains, researchers have utilized tools like Hyperledger Caliper to benchmark system performance. Gorenflo et al. [?] demonstrated that re-architecting the Fabric ordering and validation pipeline could scale throughput to 20,000 TPS, establishing an important architectural baseline. Subsequent studies provided foundational analyses of Hyperledger Fabric throughput and latency under various block sizes and endorsement policies [??]. Other implementations, such as those by [?] and [?], have successfully integrated IPFS and load balancing to achieve reliable throughput under moderate loads. Critically, the issue of MVCC transaction conflicts—where concurrent updates to the same ledger state cause silent transaction aborts—has been identified as a fundamental bottleneck in contention-heavy workloads. Wu et al. [?] proposed FabricETP, a cache-based early conflict avoidance mechanism that significantly increases throughput for concurrent conflicting transactions. Debreczeni et al. [?] provided a systematic taxonomy of MVCC conflict-addressing approaches in IEEE Access, identifying significant gaps in conflict prevention strategies and proposing a model-driven, entity-attribute partitioning approach for design-time conflict avoidance. A more recent phase-level study [?] further demonstrated that seemingly beneficial configuration changes—such as relaxing leader selection to maximize endorsement parallelism—can paradoxically introduce a modest but measurable increase in MVCC invalidations, highlighting that MVCC conflicts are sensitive to both workload characteristics and system configuration in non-obvious ways. However, none of these benchmarking or optimization efforts develop a *predictive* model capable of forecasting MVCC failure rates and throughput from static chaincode complexity features alone—the gap this study directly addresses.

2.3 Machine Learning for Blockchain Performance Prediction

As permissioned blockchain networks scale, accurately predicting their performance bottlenecks has become increasingly critical. Machine Learning (ML) models have been widely employed in broader distributed systems for anomaly detection and dynamic resource provisioning, demonstrating strong capability in capturing non-linear system behavior [**?**]. However, their application toward predicting blockchain-specific transaction failures remains nascent. Studies indicate that under heavy or highly concurrent workloads, up to 40% of all transaction failures are caused by MVCC invalidations during the validation stage—typically when multiple concurrent transactions read and try to modify the same state key before a block is finalized [**?**]. The majority of current literature combining ML with blockchain focuses either on cryptocurrency price prediction, smart contract vulnerability detection, or supply chain data integrity [**?**]. Three distinct ML research directions have recently emerged for addressing MVCC in permissioned blockchains.

2.3.1 Graph Neural Networks for Structural Conflict Detection. Transactions and world state keys form complex, interconnected dependency structures in which traditional tabular ML struggles with relational data. This has led researchers toward Graph Neural Networks (GNNs). The most prominent work is the Hyperledger Fabric-based Modified Graph Isomorphism Network (HFGIN) proposed by Alzahrani et al. [**?**], published in IEEE Access (2025). HFGIN models concurrent transactions as graph nodes and their shared read-write sets as edges, then employs a GNN to detect which transactions will trigger MVCC failures *before* they are submitted to the ordering service. By embedding HFGIN early into the EOV framework, the network predicts conflicts at the post-endorsement stage, significantly outperforming standard Graph Convolutional Networks (GCN) and Graph Isomorphism Networks (GIN)—achieving a test accuracy of 82.20%, a precision of 86.06%, an F1-score improvement of 23.37% over GCN, and computational efficiency with low inference latency under large transaction loads. HFGIN also enables parallel execution of predicted non-conflicting transactions, reducing wasted ledger bandwidth.

2.3.2 Temporal and Dynamic Workload Prediction. MVCC conflict rates are highly dynamic and correlated with transaction arrival velocity and system configuration. A phase-level benchmarking study [**?**] demonstrated that seemingly beneficial configuration changes—such as relaxing leader selection to maximise endorsement parallelism—can paradoxically introduce a modest but measurable increase in MVCC invalidations due to out-of-sync block heights, highlighting that MVCC failures are sensitive to configuration in non-obvious ways. Advanced deep learning and regression models are increasingly being paired with benchmarking platforms to monitor transient telemetry spikes in real time, forecasting load-dependent conflict thresholds to dynamically optimise block sizes or throttle submission rates for maximising successful commits.

2.3.3 ML-Driven Scheduling and Parallel Execution. Once an ML model predicts an MVCC conflict, the next frontier is using that prediction to adaptively schedule transactions. When the ML engine identifies a batch of independent, non-conflicting transactions, it flags them for aggressive parallel validation pathways [**?**]. For transactions with high MVCC conflict probability—such as hot-key wallet updates in supply chain auctions—the platform can proactively serialize them, cache block header state for rapid validation, or hold them back until the conflicting key state is fully committed.

2.3.4 Comparison: Standard vs. ML-Enhanced MVCC Handling. Table **??** summarizes the key operational differences between standard Hyperledger Fabric MVCC handling and ML-enhanced predictive approaches.

Table 1. Standard Hyperledger Fabric MVCC Handling vs. ML-Enhanced Predictive Approaches

Operational Metric	**Standard HLF MVCC**	**ML-Enhanced Prediction**
Detection Stage	Late stage: validation phase at the committing peer, post-ordering [?]	Early stage: post-endorsement or pre-ordering via graph or feature inference [?]
Computational Overhead	High; resources wasted simulating, ordering, and broadcasting transactions that ultimately abort [?]	Low; inference applied selectively, aborting or rescheduling doomed transactions early [?]
Handling Mechanism	Hard abort; client must catch the failure and manually resubmit [?]	Proactive management; transactions dynamically scheduled into parallel or serial queues based on graph structures or feature predictions [?]
Feature Requirement	None; reactive	Runtime read-write set graph (HFGIN) or static complexity features (this study)
Deployment Stage	Post-deployment only	Pre-deployment (static features) or runtime (GNN)

2.3.5 Positioning This Study. While HFGIN and similar GNN-based approaches represent state-of-the-art runtime MVCC conflict detection, they require real-time instrumentation of the transaction read-write set graph during execution, imposing runtime overhead and depending on dynamic workload data unavailable prior to deployment. In contrast, this study's XGBoost framework operates entirely from *static* chaincode complexity features (number of ledger writes, hot-key participant density) and *pre-execution* network configuration parameters (send rate, worker count), enabling offline, design-time prediction of MVCC failure rates and transaction throughput without any runtime graph construction. This makes the proposed approach uniquely suited for the *pre-deployment design decision* scenario—where a blockchain architect must evaluate whether a new chaincode function or participant configuration will introduce unacceptable MVCC failure rates before committing to production. The two approaches are thus complementary rather than competing.

To address these limitations, this study proposes a comprehensive Hyperledger Fabric-based agricultural supply chain system as a real-world testbed for MVCC performance analysis. The system integrates end-to-end traceability of crop lots, automated smart-contract-driven payments, and a decentralized auction mechanism for fair price discovery. Deploying such complex, high-frequency smart contracts inherently exposes the network to severe MVCC conflicts during competitive bidding and concurrent payment settlements. The central contribution is an XGBoost machine learning pipeline that accurately predicts both achievable throughput and stochastic MVCC failure rates from interpretable structural and operational features—providing actionable, pre-deployment diagnostics that complement runtime GNN-based detectors.

3 Methodology

This study follows a design-and-evaluation methodology. First, a Hyperledger Fabric-based agricultural supply chain platform was designed to support traceability, quality testing, auction-based price discovery, smart contract-driven payments, and packet-level consumer verification. Second, the system was implemented using JavaScript chaincode, a React.js frontend, a Node.js REST API layer, IPFS-based off-chain storage, and a multi-organization Fabric network. Finally, the system was functionally validated and then comprehensively benchmarked using Hyperledger Caliper

under a wide range of transaction loads and participant configurations to generate the empirical dataset for machine learning-based MVCC and throughput prediction.

3.1 System Overview

The proposed system is a permissioned blockchain-based platform for a high-value agricultural supply chain. It connects five stakeholder groups: farmers, aggregators, retailers, consumers, and the bank. Farmers submit crop lots with production details, and aggregators test and grade the lots for quality compliance. Approved lots are offered to retailers through a smart contract-driven bidding process. After purchase, retailers pack the lots into traceable packets, while consumers verify the packet history using the packet code. The bank creates and funds internal wallets to support smart contract-driven payments. All key events are recorded on the ledger to ensure transparency, accountability, and end-to-end traceability. This architecture—with its concurrent financial transactions, competing auction bids, and shared wallet state—creates MVCC contention patterns that serve as the primary subject of the performance analysis in this study.

3.2 Blockchain Network Architecture

The blockchain network was implemented using Hyperledger Fabric with five organizations: FarmersMSP, AggregatorsMSP, RetailersMSP, ConsumersMSP, and BankMSP. Each organization was configured with its own peer, MSP, and certificate authority. The organizations interact through a common channel, while CouchDB stores the world state. The network was created by extending the Fabric test network using customized Docker Compose, crypto-configuration, and channel configuration files. Raft was used as the ordering mechanism.

3.3 Smart Contract Workflow

The business logic was implemented as JavaScript chaincode. The bank creates and funds wallets using `createAccount`. Farmers submit lots via `createLot`, and aggregators update quality status using `testQuality`. Retailers bid via `makeOffer`, and the highest bidder executes `purchaseLot`, transferring funds automatically from their wallet to the farmer's wallet. Retailers repackage purchased lots using `createPacket`. Key features include: `createWallet()` and `depositMoney()`. Farmers submit agricultural lots using `submitProduce()`, and the testing fee is transferred to the selected aggregator. Aggregators record quality test results and grading details using `testQuality()`. Approved lots are made available for retailer offers through `makeOffer()`, and farmers accept the highest valid offer using `acceptOffer()`. Retailers then pack accepted lots into packets using `packLotIntoPackets()`, while consumers purchase packets using `purchasePacket()` and verify provenance through lifecycle query functions. The overall sequence of stakeholder actions and smart contract functions is shown in Figure **??**.

3.3.1 Traceability, Auction, and Incentive Mechanism. The traceability mechanism links each crop packet to its original lot, test result, grading details, and packing record. Testing and packing videos are stored off-chain in IPFS, while their hashes are stored on-chain for tamper-proof verification. This allows consumers to verify the product history using the packet code.

The auction mechanism supports price discovery for approved lots. Retailer offers are stored as separate composite-key records instead of being written into the lot record. This design choice is critical from an MVCC perspective: by giving each offer its own unique composite key, concurrent retailer bids no longer update the same shared lot

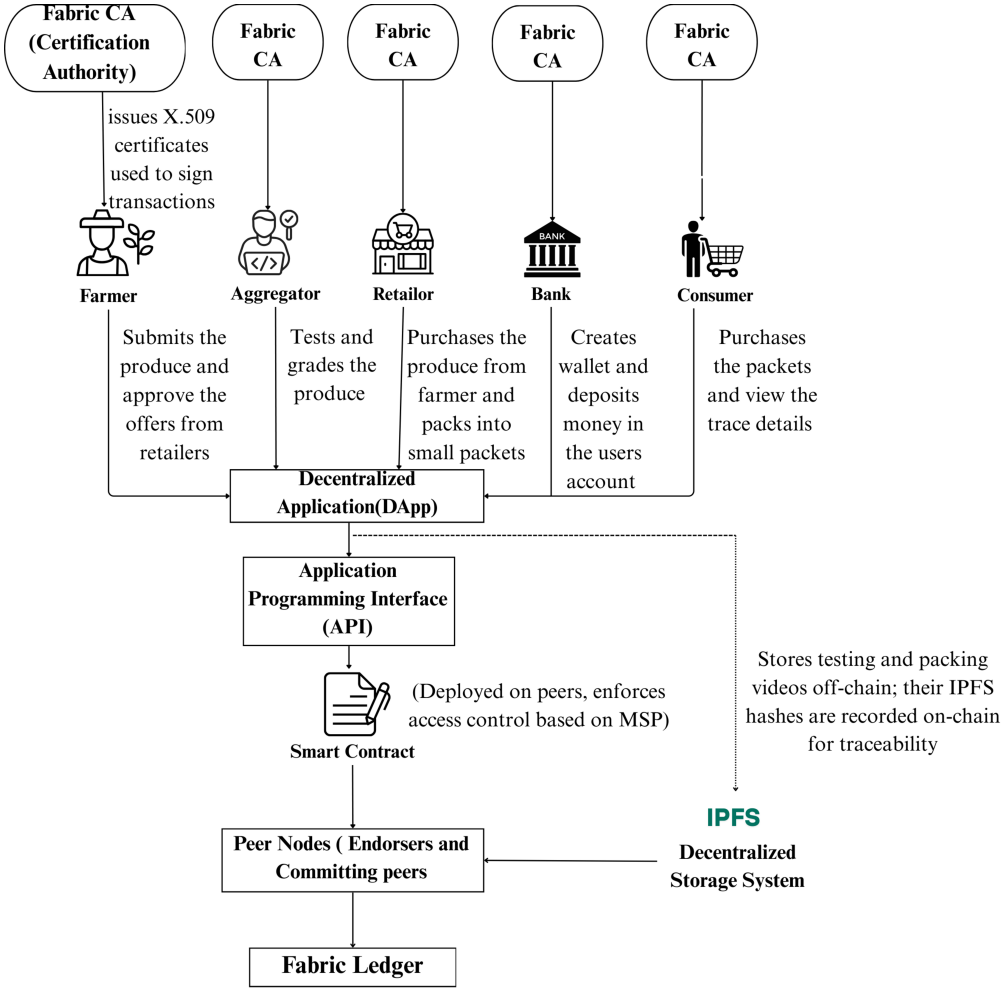

Fig. 1. System overview of the proposed agricultural supply chain platform

state, thereby reducing MVCC contention. After the farmer accepts the highest valid offer, the smart contract transfers payment from the retailer wallet to the farmer wallet and updates lot ownership.

The incentive mechanism links quality compliance with farmer reputation. Approved lots improve the farmer rating and increase visibility to retailers, while rejected lots reduce the rating. This encourages farmers to follow quality-compliant cultivation practices and improves the credibility of producers. The integration of traceability, auction-based price discovery, and quality-compliance-based incentives is illustrated in Figure **??**.

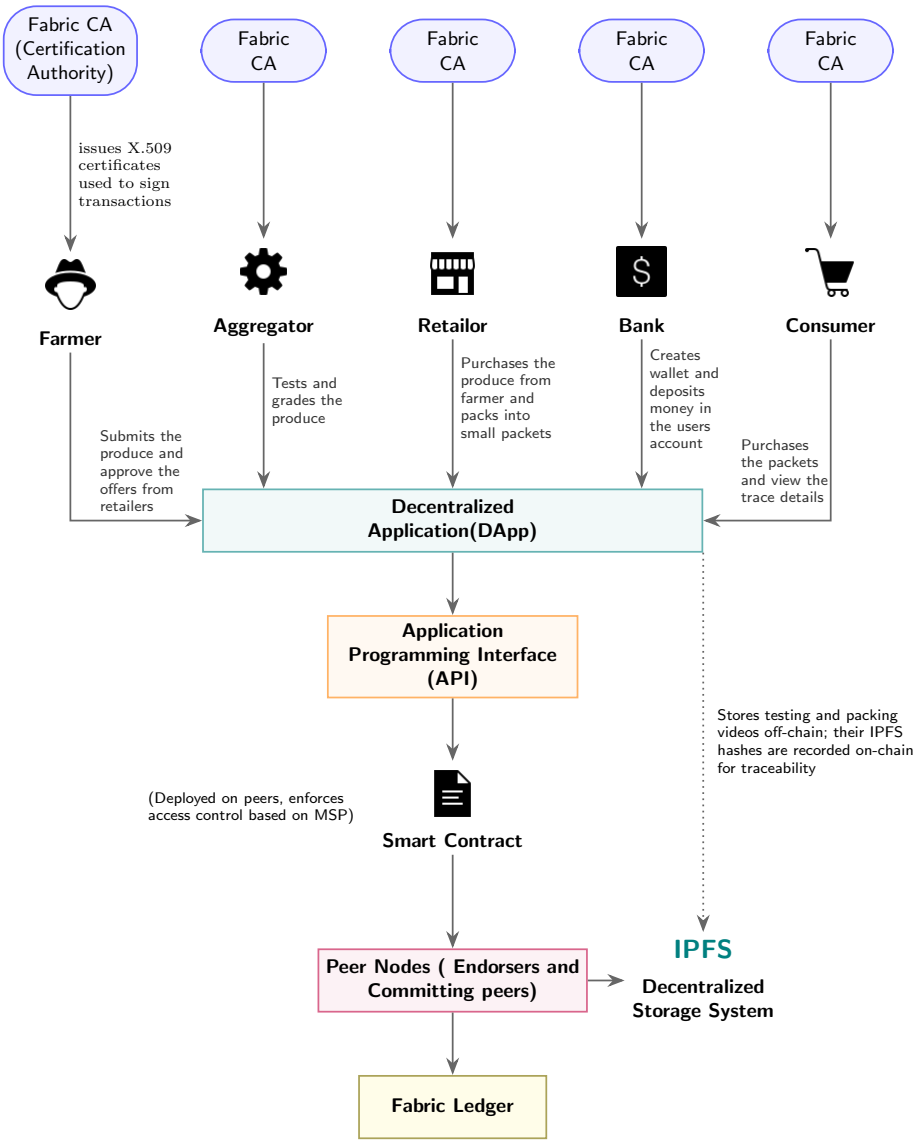

Fig. 2. Network Architecture

Figure_4.pdf

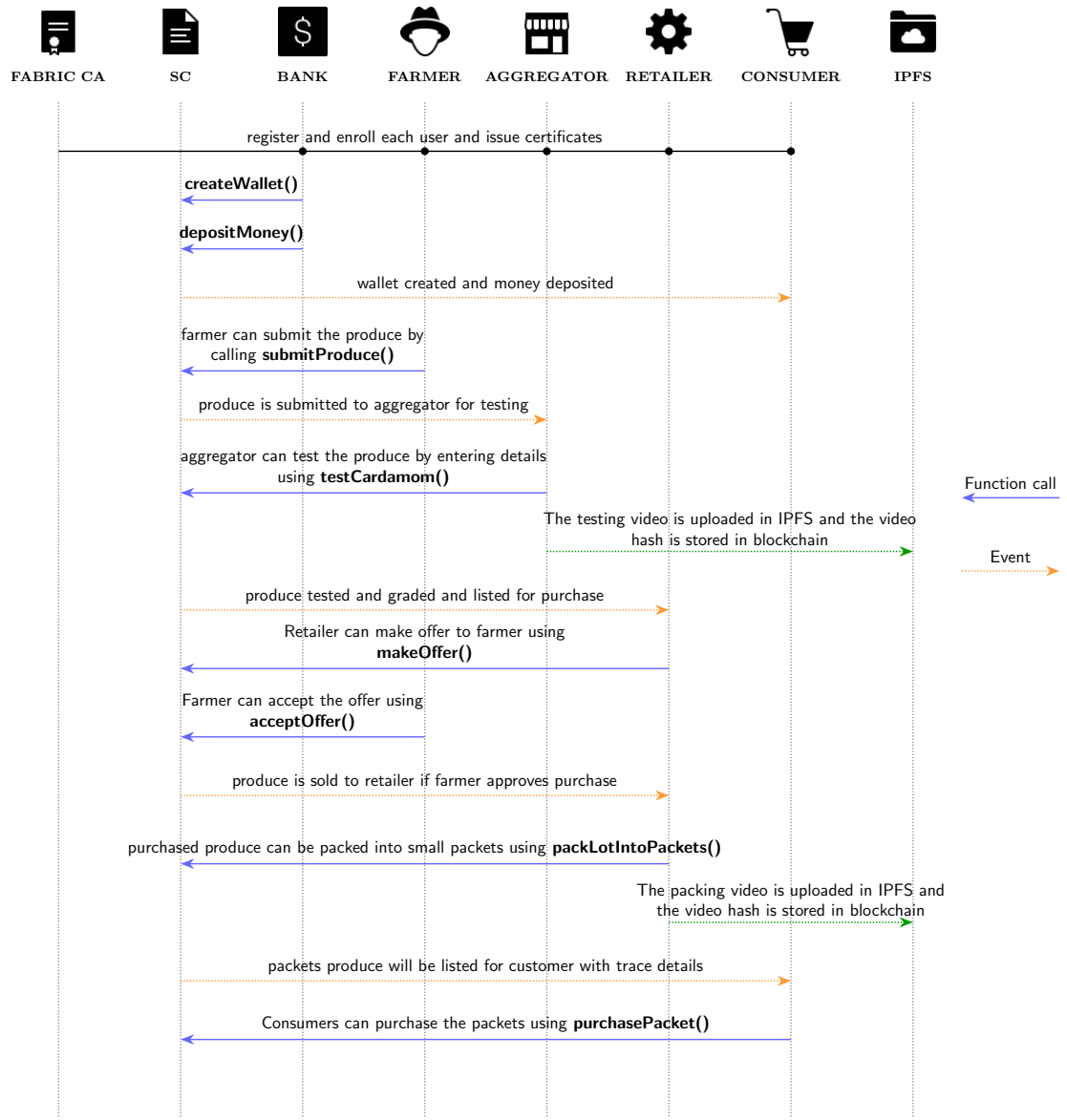

Fig. 3. Sequence of stakeholder actions and smart contract functions in the agricultural supply chain

3.4 Application Layer and IPFS Integration

A React.js web application was developed with separate dashboards for farmers, aggregators, retailers, consumers, and the bank. The frontend communicates with a Node.js backend through REST APIs. The backend invokes chaincode functions using the Fabric Gateway SDK and communicates with Fabric peers through gRPC. Large files, such as testing and packing videos, are stored off-chain in IPFS. The generated IPFS content identifier is stored on-chain with the

corresponding lot or packet record, allowing users to verify external evidence without increasing blockchain storage overhead.

3.5 MVCC Conflict Vulnerabilities

In high-throughput environments, the unoptimized chaincode is highly susceptible to Multi-Version Concurrency Control (MVCC) conflicts. Hyperledger Fabric employs an optimistic concurrency model: transactions are executed, endorsed, and ordered before validation against the ledger state. During the validation phase, if the version of a state key read during execution does not match the current committed version on the ledger, the transaction is marked invalid (MVCC `READ_CONFLICT`) and abruptly dropped. In this supply chain design, there are two primary architectural bottlenecks where these conflicts predominantly occur:

(1) **Wallet Edits (*submitProduce, acceptOffer, purchasePacket*):** Functions that facilitate financial transactions inherently invoke a shared internal `_transfer` method. This method reads the current wallet balances of the sender and receiver, computes the new balances, and executes a `putState` operation to overwrite them. When multiple participants interact with a single entity concurrently (e.g., 50 farmers simultaneously submitting produce to a single aggregator), all 50 transactions read the exact same initial version of the aggregator's wallet key. The first transaction to successfully commit will increment the ledger version of that wallet. Consequently, the remaining 49 transactions will fail validation, severely throttling throughput.

(2) **Lot Highest Offer Edits (*makeOffer*):** The auction mechanism allows multiple retailers to bid on an available agricultural lot concurrently. The `makeOffer` function reads the lot's current `highestOffer` parameter, validates if the new incoming bid is strictly greater, and overwrites the lot state. If multiple retailers bid on the exact same lot within the same block generation window, only the first endorsed transaction will successfully commit the updated lot state. The subsequent bids will encounter an MVCC conflict on the lot's shared composite key, causing those transactions to fail regardless of whether their bid values were objectively higher.

These structural vulnerabilities form the foundational basis for the machine learning predictive models developed in this study, explicitly linking function complexity and hot-key density to deterministic transaction failure rates.

4 System Testing and Benchmarking Methodology

To comprehensively evaluate the agricultural supply-chain smart contract under varying levels of concurrent traffic and participant densities, performance benchmarking was conducted using Hyperledger Caliper (v0.5.0). Caliper allows the systematic generation of synthetic workloads against the deployed network to measure throughput, latency, and resource utilization. The extracted empirical performance metrics were subsequently utilized to train an eXtreme Gradient Boosting (XGBoost) model to predict transaction failures and system throughput.

4.1 Performance Benchmarking with Hyperledger Caliper

The performance of the blockchain network was rigorously evaluated using Hyperledger Caliper, the industry-standard benchmarking tool for blockchain frameworks. The primary objective was to observe the dynamic shifts in transaction throughput and failure rates in response to increasing transaction send rates, fluctuating participant density, varying chaincode function complexity, and the resulting Multi-Version Concurrency Control (MVCC) hot-key contention.

4.1.1 Transaction Loads and Workload Functions. To capture the full spectrum of system performance—from an idle baseline to extreme network saturation—the experiments were conducted across a sequence of fixed-rate Transaction

Per Second (TPS) loads: 1, 2, 4, 8, 10, 20, 50, 100, 200, 400, and 500 TPS. The default transaction duration for each load level was maintained at 5 seconds, with specific bulk operations extended to 100 seconds to ensure steady-state measurements. The benchmarking suite tested multiple distinct chaincode functions to represent real-world supply chain workflows, including *testQuality*, *makeoffer*, *acceptoffer*, *pack*, and *submitproduce*. Each of these functions possesses a different structural complexity in terms of the number of state reads and writes required, as well as the number of participants contending for the same ledger key.

4.1.2 Strategic Participant Matrix and Dynamic Caliper Network Generation. To evaluate the impact of concurrent user scaling without exhaustively testing redundant participant combinations, a strategic participant matrix was designed. The number of participants for key roles (e.g., Farmers, Aggregators, Retailers) was scaled across predefined thresholds: 1, 2, 5, 10, 15, 20, and 24.

For functions requiring interactive multi-party dependencies (such as Farmers submitting produce to Aggregators), a two-dimensional participant matrix was generated. This approach ensured comprehensive coverage of both balanced and imbalanced participant distributions while avoiding redundant reverse pairs.

Caliper workers were dynamically allocated based on the specific workload parameters. For example, in a *submitproduce* workload configured with 5 farmers and 24 aggregators (*submitproduce_f5_a24*), Caliper explicitly spawned 5 parallel worker processes acting as independent farmers. During execution, each farmer process probabilistically targeted one of the 24 available aggregators, actively stressing the shared ledger state and inducing measurable MVCC conflicts on the specific aggregator hot-keys.

To optimize the benchmarking infrastructure, un.js dynamically generated a workload-specific Caliper network configuration file for each experimental round. This dynamic provisioning ensured that only the required organizational identities (e.g., *User1* to *User24* for Farmers and Retailers) were actively instantiated and loaded into memory, thereby isolating chaincode performance metrics from artificial client-side resource starvation.

Table 2. Benchmark Workload Configurations and Participant Mapping

Workload Function	Primary Role	Caliper Workers	Participant Target Range
submitproduce	Farmers	X (number of farmers)	Aggregators (1 to Y)
testQuality	Aggregators	N (number of aggregators)	N/A
makeoffer	Retailers	N (number of retailers)	N/A
acceptoffer	Farmers	X (number of farmers)	Retailers (1 to Y)
pack	Retailers	N (number of retailers)	N/A

4.2 Machine Learning Method and Procedure

The empirical output logs generated by Hyperledger Caliper across all benchmarking permutations were consolidated and extracted to formulate the primary dataset for the machine learning pipeline.

4.2.1 Static Analysis of Smart Contract Function Complexity. To precisely quantify the computational and state-interaction overhead of the chaincode, a custom static analysis pipeline (*analyzeComplexity*) was developed. This tool systematically parses the smart contract source code to extract deterministic operational metrics per function, including the total number of ledger reads (*getState*), ledger writes (*putState*), conditional branches, and iteration loops. Most importantly,

the tool explicitly identifies the exact composite keys targeted during write operations (MVCC Write Keys). By defining these structural metrics (detailed in Table **??**), the machine learning model can definitively correlate transaction performance with the exact number of highly contested hot-keys updated per invocation.

Table 3. Static Analysis of Smart Contract Function Complexity and MVCC Hot-Keys

Function	Total Reads	Total Writes	MVCC Write Keys (Hot-Keys)
submitProduce	3	3	Wallet Balance (fromKey, toKey)
testQuality	1	1	None
makeOffer	2	1	lot.highestOffer
acceptOffer	3	3	Wallet Balance (fromKey, toKey)
packLotIntoPackets	1	2	None
purchasePacket	3	3	Wallet Balance (fromKey, toKey)

4.2.2 Mechanics of MVCC Hot-Key Contention. In Hyperledger Fabric, Multi-Version Concurrency Control (MVCC) ensures data integrity by preventing concurrent transactions from modifying the exact same ledger state simultaneously. A "hot-key" bottleneck occurs when multiple active clients attempt to update the same state variable (key) at the exact same moment. If multiple transactions target the same state variable in the same block window, the first transaction successfully commits, while all subsequent concurrent transactions are instantly rejected by the validation peers with an MVCC read-write conflict, resulting in transaction failure.

In the context of the proposed agri-food smart contracts, these hot-key collisions mathematically dictate the failure rate and manifest in specific algorithmic patterns:

- **Wallet Balance Edits (*acceptOffer*, *purchasePacket*):** Functions that facilitate financial settlements require updating the wallet keys (e.g., `token_balance`) of both the buyer and the seller. For instance, when multiple consumers simultaneously purchase items from a highly active retailer, the retailer's single wallet key becomes a massive congestion point, triggering severe MVCC conflicts.
- **Bidding on Shared Assets (*makeOffer*):** When multiple retailers concurrently submit competitive bids on a single, highly desirable produce lot, the smart contract continuously attempts to update the `lot.highestOffer` parameter within the exact same `lotKey` state object. Consequently, only one bid can successfully commit per state version, drastically increasing the likelihood of rejection for all other concurrently executing bids.

4.2.3 Data Collection and Feature Selection. The dataset's features were carefully selected to capture the nonlinear interactions inherent in blockchain architecture that directly trigger transaction failures. Because MVCC failures in Hyperledger Fabric are predominantly caused by concurrent read/write operations on identical state keys, the selected independent variables explicitly quantified both the computational load and the specific state-write dependencies. The primary features included:

- **Transaction Send Rate (Load):** The targeted number of transactions submitted per second.
- **Number of Caliper Workers:** The number of parallel client workers actively submitting transactions to the network simultaneously, directly dictating concurrent thread pressure.
- **Participant Density:** The explicit count of active Farmers, Aggregators, Retailers, Consumers, and Bank Users interacting with the smart contract.

- **Number of Ledger Writes:** Extracted directly from the *analyzeComplexity* static analysis tool, this integer represents the absolute baseline structural complexity of the transaction. Rather than passing categorical function names, feeding the model the exact ledger write count allows it to mathematically correlate the underlying algorithmic topology to failure vulnerabilities.
- **Hot-Key Participant Count:** A derived feature representing the severity of concurrent operations on shared state keys. As identified during the functional analysis, during the *submitProduce* function, both the farmers' and aggregators' wallet accounts are overwritten continuously. During *makeOffer*, a specific lot's highest offer parameter is updated by competing retailers. Most critically, during *acceptOffer*, the wallet accounts of both the farmer and the retailer are rewritten, explicitly exposing the transaction to two distinct, highly contended hot keys. The total count of participants interacting with these precise hot-keys directly affects the probability of MVCC failure.
- **Network Metrics:** Additional network infrastructure variables, including artificially injected network latency (ms), were logged when applicable.

To provide a comprehensive dataset for the machine learning model, the benchmarking framework generated permutations across these features using the parameter ranges defined in Table **??**.

Table 4. Experimental Benchmarking Parameter Ranges

Experimental Parameter	Evaluated Range / Values
Transaction Send Rate (TPS)	1, 2, 4, 8, 10, 20, 50, 100, 200, 400, 500
Number of Caliper Workers	1, 2, 5, 10, 15, 20, 24
Participant Density Matrix	(1,1), (1,2), (1,5), (2,2), (2,5), (5,5) ... up to (24,24)
Transactions per Round	500 fixed transactions per permutation
Workload Duration	Dynamic based on TPS, with specific caps (e.g., 100s)
Network Impairments	Baseline (0ms latency)

4.2.4 Predictive Modeling and Baseline Comparison. To map the relationship between these system variables and the observed chaincode behavior, five distinct machine learning regression architectures were evaluated:

- **Linear Regression:** Baseline linear correlation model.
- **Support Vector Regressor (SVR):** Distance-based regression mapping.
- **Multi-Layer Perceptron (MLP):** A standard Neural Network architecture.
- **Random Forest Regressor:** A tree-based ensemble method.
- **eXtreme Gradient Boosting (XGBoost):** An advanced, highly optimized tree-based boosting framework.

The selection of eXtreme Gradient Boosting (XGBoost) as the primary predictive architecture was driven by the unique failure mechanics of Hyperledger Fabric. Blockchain performance degradation—specifically MVCC read-write conflicts and throughput saturation—does not scale linearly. A network remains perfectly stable until a specific structural limit is hit (e.g., a critical mass of concurrent hot-key collisions or resource exhaustion), at which point performance drops off a severe "cliff" (jumping abruptly from 0% to high failure rates). Linear mapping models like Linear Regression and Support Vector Regression (SVR) are fundamentally incapable of modeling these strict, sudden thresholds. While Neural Networks (MLPs) can model non-linear boundaries, they inherently blur discrete integer transitions

and lack interpretability. Conversely, tree-based ensemble methods, particularly XGBoost, create distinct conditional splits (e.g., IF Hot-Keys > 2 AND Load > 300 → Failure = 85%), perfectly mimicking the conditional transaction validation logic of blockchain state machines. Furthermore, XGBoost natively calculates information gain (Feature Importance), allowing for the extraction of mathematical proof regarding which structural parameters act as the primary bottlenecks, making it the optimal and most interpretable choice for this study.

Crucially, rather than predicting the absolute number of failed transactions (which is artificially biased by the total number of transactions sent), the models were trained to predict the precise *Failure Rate* (the percentage of failed transactions over the total submitted), alongside the successfully achieved system *Throughput*.

5 Results and Discussion

5.1 Functional Validation

Functional validation was performed using both manual and automated testing. Manual testing was conducted through the web interface, REST API calls, and CLI commands to verify realistic stakeholder interactions. Automated scripts were then used to execute repeated transaction flows and confirm stable ledger updates.

The validation covered wallet creation and funding, produce submission, quality testing and grading, offer creation and acceptance, packet generation, consumer purchase, and provenance queries. Each transaction was followed by query operations to confirm correct state transitions, ownership updates, wallet changes, lot and packet status updates, and metadata consistency. Role-based access was tested using MSP identifiers to ensure that restricted functions could be invoked only by authorized stakeholders.

IPFS integration was validated by checking whether testing and packing video hashes were correctly stored on-chain and matched the corresponding off-chain files. These tests confirmed that the system preserved traceability and data consistency across the complete cardamom supply chain workflow.

5.2 Smart Contract Execution Summary

Table **??** summarizes the main smart contract functions and their validated outcomes. The validation confirms that the implemented chaincode correctly supports wallet operations, produce submission, quality testing, auction-based offer handling, packetization, consumer purchase, and traceability verification.

Overall, the validation confirmed that the system executed the intended stakeholder workflows correctly and maintained consistent ledger, wallet, and traceability records.

5.3 Function-wise Write Performance

Each core write function was evaluated independently under varying send rates using Hyperledger Caliper. The tested functions included *submitProduce()*, *testQuality()*, *makeOffer()*, *acceptOffer()*, *packLotIntoPackets()*, and *purchasePacket()*. The objective was to contextualize the empirical throughput and failure rates prior to machine learning analysis.

Table **??** isolates the baseline network environment (0ms induced latency) to demonstrate how increasing concurrent load and client scaling distinctly affects different smart contract topologies.

Conversely, Table **??** illustrates the severe performance degradation introduced by wide-area network latency (10ms and 20ms) on a single function (*submitproduce*).

Table 5. Smart Contract Execution Validation

Function	Validated Outcome
`createWallet()`, `depositMoney()`	Wallets were successfully created and token balances initialized for all stakeholders.
`submitProduce()`	Farmers submitted produce lots; testing fees were correctly deducted and transferred to aggregators.
`testQuality()`	Test results and grading data were recorded; lot status updated based on quality score evaluation.
`makeOffer()`, `acceptOffer()`	Retailers submitted offers; farmers accepted valid offers and payments were transferred accordingly.
`packLotIntoPackets()`	Retailers generated traceable packets (100g–1kg) with associated metadata stored on-chain.
`purchasePacket()`	Consumers purchased packets and corresponding financial transactions were successfully executed.

Table 6. Comprehensive Baseline Network Performance Profiles (0ms Latency)

Function	Caliper Workers	Hot-Keys	Target Load (TPS)	Throughput (TPS)	Failure Rate
submitproduce	1	2	500	161.2	91.6%
submitproduce	10	20	500	278.2	98.1%
submitproduce	24	48	500	241.8	97.0%
testQuality	1	0	500	132.2	0.2%
testQuality	10	0	500	281.6	0.0%
testQuality	24	0	500	199.0	0.0%
makeoffer	1	1	500	235.9	46.6%
makeoffer	10	10	500	357.1	48.0%
makeoffer	24	24	500	341.8	48.0%
acceptoffer	1	2	500	0.0	0.0%
acceptoffer	10	20	500	39.6	87.6%
acceptoffer	24	48	500	38.9	88.7%
pack	1	0	500	0.0	0.0%
pack	10	0	500	0.0	0.0%
pack	24	0	500	0.0	0.0%
purchase	1	2	500	25.7	0.2%
purchase	10	20	500	243.0	99.0%
purchase	24	48	500	208.5	96.8%

At 500 TPS, *makeOffer()* achieved the best scalability among the tested write functions, maintaining very low average latency and reaching a maximum throughput of 416.3 TPS. This result reflects its lightweight design and reduced shared-state updates.

In contrast, more contention-prone operations exhibited higher latency and failure rates. The *packLotIntoPackets()* function reached a throughput ceiling of about 140 TPS because each transaction creates multiple packet records, increasing endorsement and commit overhead. The *purchasePacket()* function achieved high throughput but produced

Table 7. Impact of Network Latency on Throughput and Reliability (*submitproduce*)

Latency	Caliper Workers	Target Load (TPS)	Throughput (TPS)	Failure Rate
10ms	10	1	1.9	30.0%
10ms	5	500	244.0	99.4%
20ms	10	1	1.9	35.0%
20ms	5	500	230.2	99.5%

higher failure counts because multiple clients attempted to purchase the same packet concurrently, causing MVCC conflicts on the shared packet asset.

5.4 Phase 2: Retrieval Function Benchmarking

The second phase evaluated read-only functions, including *getAllProduce()*, *getAllPackets()*, and *getPacketLifecycle()*. These functions do not modify ledger state but retrieve data using different access patterns. The *getAllProduce()* and *getAllPackets()* functions access the world state through composite-key scans and filtering, while *getPacketLifecycle()* reconstructs provenance using transaction-history traversal.

Table **??** shows that *getPacketLifecycle()* remained stable from 50 to 500 TPS, with no failures and very low average latency. Throughput closely followed the configured send rate, reaching 495.8 TPS at 500 TPS. This indicates that provenance queries can scale efficiently when history traversal is limited and no world-state updates are involved.

Table 8. Performance Results for `getPacketLifecycle` (Payload: 2908 bytes) under Varying Load

Function	TPS	Total Tx	Failed	Send Rate	Max Lat (s)	Min Lat (s)	Avg Lat (s)	Throughput
getPacketLifecycle	50	5000	0	50.0	0.06	0.00	0.01	50.0
getPacketLifecycle	100	4995	0	99.9	0.02	0.00	0.01	99.9
getPacketLifecycle	200	5000	0	199.8	0.50	0.01	0.01	199.8
getPacketLifecycle	300	5000	0	299.4	0.03	0.01	0.01	299.3
getPacketLifecycle	400	5000	0	377.8	1.19	0.00	0.01	373.6
getPacketLifecycle	500	5000	0	496.2	0.10	0.00	0.02	495.8

Table **??** shows that world-state retrieval performance is strongly affected by payload size. Smaller payloads remained stable at lower and moderate loads, while larger payloads caused sharp latency growth, reduced throughput efficiency, and higher failure rates. This behaviour is mainly due to composite-key scanning, filtering, data serialization, and network transfer overhead. The results show that payload size is a major bottleneck for large read queries in Fabric-based supply chain applications.

The table presents the performance of both the *getAllPackets()* and *getProduce()* functions under varying payload sizes at a representative 100 TPS load. The results demonstrate a strong dependency of query performance on the volume of data retrieved.

For smaller payload sizes, the functions maintain stable performance with low latency and no failures. However, as the payload size increases significantly, performance degrades even at this moderate load level. Latency increases sharply and failure rates become substantial. Throughput drops considerably, reflecting the combined impact of expensive world-state traversal, large data serialization, and increased network transfer overhead. The larger response size prolongs request processing time, leading to accumulation of concurrent requests in the system.

This behaviour is primarily attributed to the increasing cost of world-state traversal, data aggregation, and response serialization as payload size grows. Since these functions rely on composite key scans and filtering operations over the world state, larger result sets significantly increase computational and I/O overhead. These results highlight that payload size is a dominant factor influencing read performance and that large-scale data retrieval can become a critical bottleneck in blockchain-based applications.

Table 9. Performance Results for Retrieval Functions at 100 TPS under Varying Payload Sizes

Function	Payload	TPS	Success	Failed	Send Rate	Avg Lat (s)	Throughput
`getAllPackets`	66 KB	100	5000	0	100.0	0.01	99.9
`getAllPackets`	330 KB	100	5000	0	100.0	10.81	77.4
`getAllPackets`	660 KB	100	1049	3941	99.8	17.10	74.5
`getProduce`	94.3 KB	100	4996	0	99.9	0.01	99.8
`getProduce`	471.5 KB	100	4215	785	100.1	32.32	55.2
`getProduce`	943 KB	100	427	4573	100.0	16.37	93.3

5.5 Impact of Network Latency

To examine the effect of network conditions, additional experiments were conducted by introducing controlled impairments between Docker containers: 50 ms fixed latency. Since the impact of network variability is more visible at higher loads, Table **??** reports the results at 500 TPS.

Table 10. Performance Comparison of Smart Contract Functions at 500 TPS under Network Impairment

Function	TPS	Success	Failed	Send Rate	Max Lat (s)	Min Lat (s)	Avg Lat (s)	Throughput
submitProduce	500	4996	4	482.8	15.99	0.93	12.38	191.8
testQuality	500	4990	3	482.4	15.70	0.67	11.02	193.4
makeOffer	500	5880	8	443.8	5.63	0.00	1.79	368.5
acceptOffer	500	4994	6	472.0	14.03	1.38	12.07	212.8
packLotIntoPackets	500	4993	7	590.0	70.15	19.83	62.74	70.51
purchasePacket	500	3346	1654	475.4	12.56	1.05	10.80	220.6
getProduce (13.2 KB)	500	804	4196	476.0	294.15	2.12	186.46	16.6
getAllPackets (66 KB)	500	1385	3613	483.5	293.78	1.05	223.76	16.5

Table **??** shows that network impairment substantially increases latency and reduces throughput, especially for complex write operations and large retrieval queries. Functions such as *submitProduce()*, *testQuality()*, and *acceptOffer()* show higher average latency because endorsement, ordering, and commit phases are all affected by communication delay. The *packLotIntoPackets()* function is affected most among write operations due to write amplification from multiple packet records.

The *makeOffer()* function remains comparatively stable, with lower latency and higher throughput, because it performs lightweight ledger updates. In contrast, *purchasePacket()* shows higher failures because network delay increases the time window for concurrent access to the same packet asset, thereby increasing MVCC conflicts.

Retrieval functions show the strongest degradation under impaired network conditions. Both *getProduce()* and *getAllPackets()* involve larger response payloads, making them more sensitive to latency and peer-side request accumulation. The results indicate that read-path scalability is constrained by both payload size and request concurrency.

Overall, the experiment confirms that network delay has a stronger impact on data-intensive and write-heavy functions than on lightweight transaction functions.

5.6 Runtime Resource Utilization

Table **??** summarizes CPU, memory, and network I/O usage for representative write and retrieval functions. Write-intensive functions generally show increased CPU utilization at higher loads, especially *packLotIntoPackets()* and *acceptOffer()*, because they involve multiple state updates and ledger writes. In contrast, *purchasePacket()* maintains relatively low CPU and memory usage, although its failures are mainly related to MVCC conflicts rather than resource exhaustion.

Retrieval functions show moderate CPU usage but higher network I/O, particularly for *getAllPackets()* and *getProduce()*, because they return larger payloads from the ledger. Memory usage remains stable across all functions, indicating that no abnormal memory growth or resource exhaustion occurred during benchmarking. Overall, the results suggest that the observed throughput and latency variations are mainly caused by transaction complexity, payload size, and concurrency effects rather than insufficient system resources.

Table 11. Runtime Resource Utilization of Smart Contract and Retrieval Functions at 500 TPS Load

Function	TPS	CPU Avg (%)	CPU Peak (%)	Mem Avg (MB)	Net I/O (MB)
submitProduce	500	26.58	82.8	369.37	7759.74
testQuality	500	27.44	78.5	208.56	5865.42
acceptOffer	500	28.86	81.7	371.74	8400.26
makeOffer	500	25.54	86.0	297.45	2681.62
packLotIntoPackets	500	38.19	85.0	379.52	7017.50
purchasePacket	500	9.96	79.3	243.63	2577.23
getAllPackets	500	21.58	69.2	340.15	6974.11
getProduce	500	12.04	51.1	305.59	7657.60
getPacketDetails	500	8.43	49.3	231.02	2586.68

5.7 Optimization to Reduce MVCC Conflicts

MVCC conflicts are a major source of transaction failure in Hyperledger Fabric when concurrent transactions update the same ledger key. In the unoptimized design, wallet transfers directly overwrote shared wallet-balance keys, while offer handling modified the same lot record during *makeOffer()*. These read–modify–write patterns created shared-key contention under concurrent workloads.

To reduce these conflicts, the chaincode was redesigned using append-only records and composite keys. Wallet transfers were recorded as separate `walletTx` debit and credit entries instead of overwriting wallet balances. Retailer offers were also stored as independent `offer` assets using composite keys rather than being embedded in the lot record. This is achieved by moving the `getWalletBalance()` check outside the transfer function and the `getHighestOffer()` check outside `makeOffer()`. These validations need to be checked in the backend rather than in the chaincode. As a result, concurrent transactions generated unique ledger entries instead of repeatedly modifying shared state.

Table **??** compares the unoptimized and optimized chaincode at 200 TPS. The optimized design substantially reduced transaction failures. For example, *acceptOffer()* improved from 3963 failures to zero, while *submitProduce()* improved

from 3827 failures to zero. The *makeOffer()* function also eliminated failures after separating offers from the lot record. Although *purchasePacket()* still recorded 866 failures, this was mainly due to application-level contention where multiple clients attempted to purchase the same packet. Overall, the results confirm that append-only composite-key records significantly improve concurrency stability in the proposed Fabric system.

Table 12. Performance Comparison of Unoptimized and Optimized Chaincode at 200 TPS

Function	TPS	Success	Fail	Send Rate	Max Lat (s)	Min Lat (s)	Avg Lat (s)	Throughput
Unoptimized Chaincode								
acceptOffer	200	1037	3963	200.1	2.20	0.08	0.99	199.8
makeOffer	200	4630	380	200.4	2.05	0.00	0.05	185.5
packLotIntoPackets	200	5000	0	200.0	13.91	0.25	12.79	109.1
purchasePacket	200	1235	3765	199.4	2.84	0.05	0.10	179.4
submitProduce	200	1173	3827	199.7	0.91	0	0.10	199.2
testQuality	200	5000	0	199.9	0.73	0.04	0.10	199.5
Optimized Chaincode								
acceptOffer	200	5000	0	199.2	1.49	0.00	0.12	198.7
makeOffer	200	5010	0	199.6	2.04	0.00	0.05	185.0
packLotIntoPackets	200	5000	0	199.5	17.17	0.19	10.73	118.9
purchasePacket	200	4134	866	200.0	2.05	0.04	0.10	185.2
submitProduce	200	5000	0	200.0	1.11	0.00	0.07	199.6
testQuality	200	5000	0	199.8	0.15	0.00	0.08	199.4

5.8 Practical Implications and Limitations

The system is suitable for practical agricultural supply chain scenarios where transactions are usually asynchronous and occur at lower rates than the stress-test conditions. Under realistic operating loads, the system can support traceability, quality verification, auction-based price discovery, and smart contract-driven payments.

However, the reported results should be interpreted as controlled laboratory measurements rather than production-scale throughput values. The experiments were conducted in a single-host Docker environment using WSL2, which does not fully capture wide-area network latency, geographically distributed endorsement, or multi-host ordering overhead. Future work should evaluate the system in a distributed deployment and optimize read-heavy functions through pagination, indexing, caching, and improved query design.

5.9 Machine Learning Predictive Insights

By training the machine learning pipeline on the empirical dataset, the models provided mathematical quantification of the underlying structural bottlenecks causing MVCC read-write conflicts. As detailed in the methodology, the models were blind to the categorical smart contract names and instead evaluated raw network load, client scaling, and the static *Number of Ledger Writes* extracted via *analyzeComplexity*.

5.9.1 Baseline Model Comparison. To validate the hypothesis that MVCC conflicts scale non-linearly and exhibit strict threshold "cliffs" based on structural topology, five regression architectures were evaluated. As shown in Table **??**, the tree-based ensemble methods (XGBoost and Random Forest) drastically outperformed linear mapping models (Linear

Regression, SVR) and the Neural Network (MLP). Tree-based models are uniquely capable of isolating distinct topological profiles by forming non-linear decision branches, making them the optimal choice for modeling blockchain performance limits.

Table 13. Comparative Performance of Machine Learning Models (Failure Rate Prediction)

Machine Learning Architecture	R^2 **Score**	**Mean Absolute Error (MAE)**
eXtreme Gradient Boosting (XGBoost)	0.941	4.2%
Random Forest Regressor	0.768	10.0%
Multi-Layer Perceptron (Neural Network)	0.689	15.7%
Linear Regression	0.595	19.1%
Support Vector Regressor (SVR)	0.472	22.0%

XGBoost was selected as the definitive modeling framework for the remainder of the study because it achieved the highest predictive test accuracy (R^2 = 0.82 for failure prediction). Additionally, XGBoost's sequential boosting architecture is mathematically sharper at extracting Information Gain (Feature Importance) from highly tabular, integer-based structural metrics compared to Random Forest's independent bagging approach.

5.9.2 Identification and Removal of Proxy Variables. During initial modeling, the payload size (in bytes) of each transaction was included as an independent variable. Anomalously, it absorbed 85.3% of the predictive importance, entirely suppressing the true architectural metrics. Statistical evaluation using Normalized Mutual Information (NMI) revealed that payload size acted as a near-perfect proxy identifier (NMI = 0.965) for the categorical smart contract name, as each function possessed a highly distinct payload byte footprint.

To ensure the generalized model mathematically evaluated the true architectural bottlenecks rather than exploiting a categorical shortcut, payload size was excluded from the final training dataset. Subsequently, the baseline algorithmic complexity (*Number of Ledger Writes*) emerged as the undisputed structural fingerprint, absorbing 81.2% of the predictive variance.

5.9.3 Feature Importance and Architectural Bottlenecks. A critical goal of the predictive modeling was to empirically define the catalyst of MVCC failures. Traditional blockchain literature often assumes that network load directly triggers transaction failure. However, utilizing the XGBoost Feature Importance metric (Information Gain), the model overwhelmingly determined that **the static complexity of the code dictates the failure rate.**

When calculating the exact percentage of failed transactions, XGBoost distributed the predictive weights as follows:

- **Hot-Key Participant Density:** 75.0%
- **Number of Caliper Workers:** 9.6%
- **Number of Ledger Writes:** 8.0%
- **Transaction Send Rate (Load):** 7.4%

These results prove mathematically that pushing higher network load (TPS) into the system only accounted for 7.4% of the failure rate variance. Instead, the architectural density of MVCC hot-keys (the exact algorithmic collision probability space determined by multiplying the function's static hot-keys with concurrent active clients) acted as the dominant catalyst, absorbing a massive 75.0% of the predictive importance.

Ultimately, this finding proves that MVCC failures in Hyperledger Fabric are fundamentally driven by state-contention architecture (hot-keys), not merely by network traffic.

5.9.4 Predicting Throughput Constraints. Conversely, when predicting *successful transaction throughput* (TPS), the model identified a completely different set of structural bottlenecks. The XGBoost throughput model (which achieved an R^2 of 0.84) distributed the predictive weights as follows:

- **Transaction Send Rate (Load):** 52.9%
- **Number of Ledger Writes:** 30.0%
- **Hot-Key Participant Density:** 14.4%
- **Number of Caliper Workers:** 2.7%

These results confirm the logical scaling of blockchain systems. Throughput is primarily driven by the raw volume of transactions being pushed into the network (Load). However, the absolute processing ceiling before the network saturates is heavily constrained by the static complexity of the code—specifically, the *Number of Ledger Writes*, which acts as the primary computational bottleneck for successful throughput (30.0%). Unlike transaction failures, successful throughput is only mildly degraded by MVCC hot-key collisions (14.4%), as hot-keys predominantly dictate the failure rate rather than the processing speed of successful transactions.

6 Conclusion

This study set out to address a fundamental challenge in the practical deployment of permissioned blockchain systems: the difficulty of predicting and diagnosing non-linear Multi-Version Concurrency Control (MVCC) failures before they cause catastrophic throughput degradation in production environments. Using a Hyperledger Fabric-based agricultural supply chain system as a real-world empirical testbed, we developed and validated a machine learning framework—based on eXtreme Gradient Boosting (XGBoost)—that accurately forecasts both MVCC failure rates and transaction throughput from interpretable structural and operational features.

The supply chain system itself delivers a comprehensive permissioned blockchain platform for end-to-end traceability, smart contract-driven financial settlements, and decentralized auction-based price discovery. The system leverages a tamper-proof shared ledger to immutably record all critical lifecycle events—produce submission, quality testing outcomes, grading results, and financial transactions—ensuring transparency, accountability, and trust among all stakeholders: farmers, aggregators, retailers, banks, and consumers. The blockchain-based auction mechanism, in which each retailer bid is stored as an independent composite-key record, prevents the common MVCC hot-key bottleneck on the shared `highestOffer` field and ensures competitive, transparent price discovery. Smart contract-driven payments eliminate intermediaries, reduce settlement delays, and provide verifiable financial trails, while IPFS-based off-chain storage keeps blockchain overhead manageable.

The performance analysis, conducted via Hyperledger Caliper across 11 send-rate levels and a comprehensive participant matrix, revealed severe MVCC-induced throughput degradation in functions with shared wallet keys (*submitProduce*, *acceptOffer*, *purchasePacket*) and shared lot state (*makeOffer*). Critically, the XGBoost predictive model demonstrated that **MVCC hot-key participant density** is the dominant predictor of failure rate (75.0% feature importance), far exceeding the contribution of raw network load (7.4%). Conversely, for throughput prediction, the **number of ledger writes** (structural complexity) was the primary constraint (30.0%), with send rate contributing 52.9%. These findings mathematically prove that MVCC failures are fundamentally an architectural phenomenon driven by state-contention topology, not merely by network traffic volume.

The results confirm that append-only, composite-key-based chaincode designs substantially reduce MVCC conflicts, and that the XGBoost model provides an accurate, interpretable diagnostic tool for identifying which chaincode

functions and participant configurations are at highest risk. Looking ahead, future work will extend this predictive framework to distributed multi-host deployments with wide-area network conditions, explore dynamic chaincode optimization strategies guided by model predictions, and integrate real-time IoT sensor data streams for objective quality grading. The supply chain finance layer can be extended with weather-indexed smart contract insurance and credit access mechanisms leveraging verified on-chain transaction histories, further strengthening the financial resilience and scalability of agri-food blockchain deployments.

Data availability

Data will be made available on request.

Acknowledgements

This work was supported by **National Institute of Technology Calicut (NIT Calicut)**.

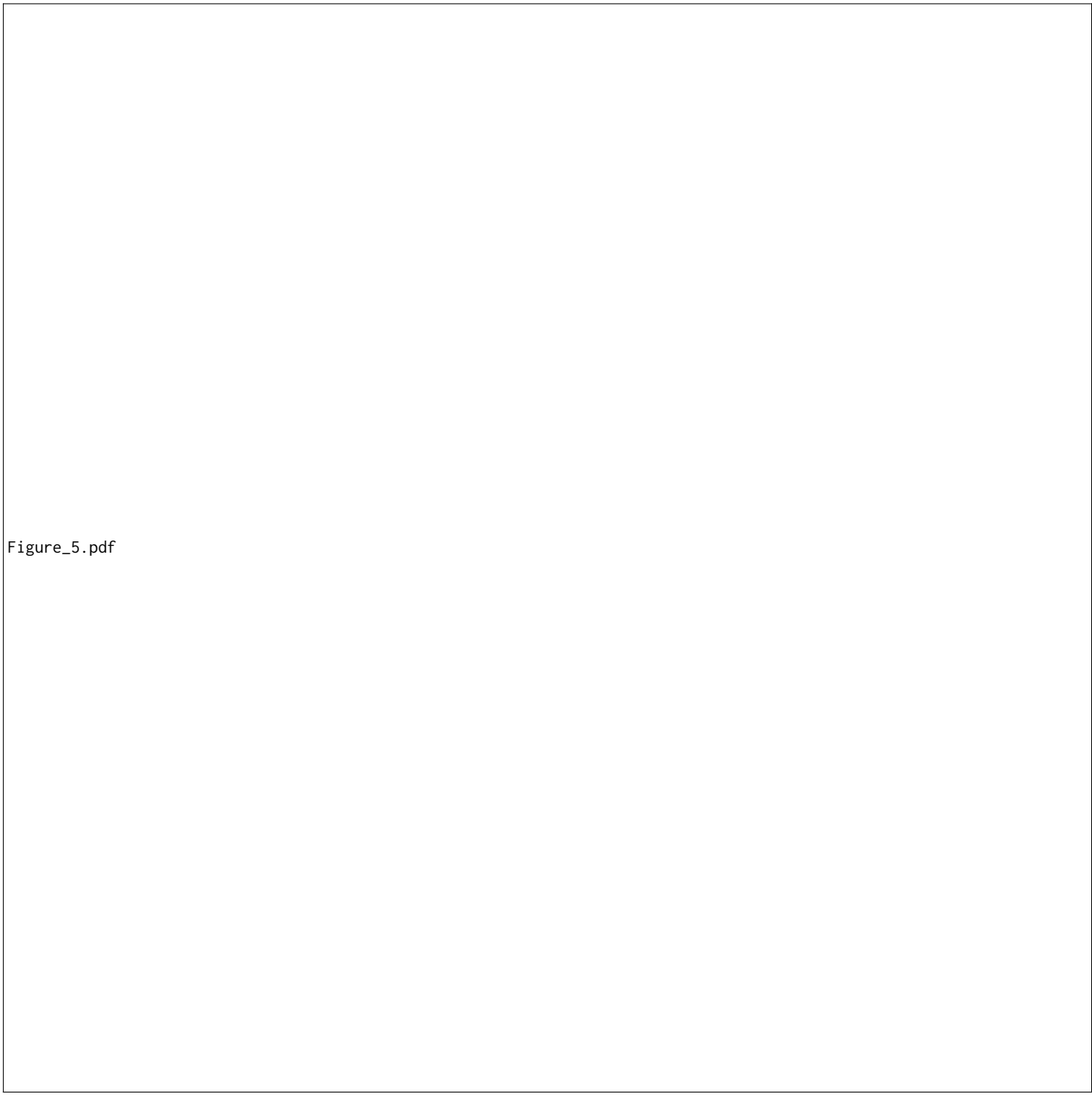

Fig. 5. Interaction between Frontend, Fabric SDK, and the Hyperledger Fabric Network

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.